

XianzhiKrylov

用户手册

C API 使用与参考

版本：26.44

适用平台：NVIDIA、Hygon、AMD、Intel、Moore Threads、Lisuan、MetaX

修订日期：2026 年 10 月 27 日

目录

1 简介 **1**
1.1 支持范围 . . . 1
1.2 文档约定 . . . 1
1.3 版本说明 . . . 1
1.4 交付物 . . . 2

2 安装与快速开始 **3**
2.1 运行与编译环境 . . . 3
2.2 解压下载文件 . . . 3
2.3 配置运行路径 . . . 3
2.4 配置 License . . . 4
2.5 编译与运行示例 . . . 4
2.6 应用链接 . . . 4
2.7 CMake 接入 . . . 4

3 通用说明 **5**
3.1 句柄与状态 . . . 5
3.2 执行阶段 . . . 5
3.3 内存所有权与生命周期 . . . 6
3.4 并发使用 . . . 6
3.5 配置与复用 . . . 6
3.6 时间统计 . . . 6

4 数据类型与存储格式 **7**
4.1 不透明对象 . . . 7
4.2 数值类型 . . . 7
4.3 矩阵描述 . . . 7
4.4 稀疏存储格式 . . . 8
4.5 向量与多右端项 . . . 8
4.6 Double-Double . . . 8
4.7 检查清单 . . . 9

5 函数参考 **10**
5.1 `xianzhikrylov_create()` . . . 10
5.2 `xianzhikrylov_config_set()` . . . 10
5.3 `xianzhikrylov_matrix_create()` . . . 11
5.4 `xianzhikrylov_vector_create()` . . . 11
5.5 `xianzhikrylov_multiple_vector_create()` . . . 12
5.6 `xianzhikrylov_execute()` . . . 13
5.7 `xianzhikrylov_print_info()` . . . 13
5.8 `xianzhikrylov_matrix_destroy()` . . . 13
5.9 `xianzhikrylov_vector_destroy()` . . . 14
5.10 `xianzhikrylov_destroy()` . . . 14

6 使用示例 **15**
6.1 快速上手 . . . 15

6.2 JSON 配置 . . . 15
6.3 常用配置：FGMRES + ILU(0) . . . 16
6.4 分阶段执行 . . . 16
6.5 相同稀疏结构，矩阵数值改变 . . . 17
6.6 同一矩阵，连续更新右端项 . . . 17
6.7 提供非零初值 . . . 17
6.8 多右端项 . . . 17
6.9 外部 AMG 预条件器 . . . 18

7 错误处理与结果验证 19
7.1 错误状态 . . . 19
7.2 常见集成问题 . . . 19
7.3 数值精度与残差 . . . 19
7.4 问题报告 . . . 20

附录 A 示例输入文件与随包样例 21
A.1 随包样例 . . . 21
A.2 矩阵：XKPAT1 模式格式 . . . 21

附录 B 文档与支持 23
B.1 交付文件 . . . 23
B.2 技术支持 . . . 23

附录 C 版本日志 24
XianzhiKrylov 26.44 . . . 24
XianzhiKrylov 26.43 . . . 24
XianzhiKrylov 26.42 . . . 24

1 简介

XianzhiKrylov 是面向 GPU 的稀疏线性系统迭代求解库，通过统一的 C 接口求解

$$Ax = b,$$

其中 A 为 $n \times n$ 稀疏方阵，b 为右端项，x 为待求解向量。应用创建矩阵和向量对象，将该组对象提交给计算句柄，再按分析、数值准备、求解等阶段执行。算法设置由 JSON 文件选择。

不同 GPU 厂商的发行包使用同一套 C 头文件、枚举和调用顺序。应用链接目标平台对应的动态库，不需要在代码中传入后端参数。

1.1 支持范围

能力	说明
迭代方法	CG、PCG、FCG、BiCG、BiCGStab、GMRES、FGMRES、GCR 及发行包开放的扩展方法。
预条件类别	Jacobi、ILU(0)、IC(0)、块 Jacobi、DILU、DIC、多色块预条件、HSGS、SSOR 等。
外部预条件器	发行包可经 JSON 选择 LinglongAMG 等外部 provider。
稀疏格式	CSR、BSR、MSR 和 LDU。
数值类型	FP64；部分发行包提供 FP32、complex FP64、complex FP32 和 Double-Double。
输入位置	主机内存和由调用方管理的设备内存。
右端项布局	RHS-major 或 row-major，支持一个或多个右端项。
并行方式	一个计算句柄绑定一个可见 GPU；独立系统使用独立句柄。

方法、预条件器、矩阵格式和精度之间存在组合约束。发行包中的 `samples/` 目录列出了该包实际开放的组合。应用应使用与 CPU 架构、glibc、GPU 和厂商工具包相匹配的发行包。

1.2 文档约定

等宽字体表示函数、类型、枚举、字段、命令和路径。代码中的 `XIANZHIKRYLOV_` 常量应完整书写。

本文中的“分析”只使用矩阵结构信息；“数值准备”使用矩阵数值建立预条件器；“求解”执行 Krylov 迭代并将结果写入调用方提供的解对象。

接口返回状态与数值正确性是两个独立问题。`XIANZHIKRYLOV_CONVERGED` 表示库按配置的停止条件完成了求解；应用还应使用原始 A 和 b 独立计算残差。

1.3 版本说明

本手册对应公开发行版本 26.44，采用 ABI 3 和下列十函数接口：

```
xianzhikrylov_create
xianzhikrylov_config_set
xianzhikrylov_matrix_create
xianzhikrylov_vector_create
xianzhikrylov_multiple_vector_create
xianzhikrylov_execute
```

```
xianzhikrylov_print_info
xianzhikrylov_matrix_destroy
xianzhikrylov_vector_destroy
xianzhikrylov_destroy
```

应用需使用随发行包提供的头文件重新编译。安装的厂商发行包决定运行后端。

1.4 交付物

发行包中的主要文件如下：

```
XianzhiKrylov/
  README.md
  THIRD_PARTY_NOTICES.txt
  include/
    XianzhiKrylovAPI.h
    XianzhiKrylovData.h
  lib/
    libXianzhiKrylov.so
    ...
  samples/
    README.md
    common.mk
    01_cg/
    ...
```

授权文件下载后放在包根目录并命名为 `license.key`。软件包与授权文件可从 https://store.solved-tech.com/ 获取。

2 安装与快速开始

本章按“确认环境、解压软件包、配置运行路径、配置 License、运行样例、接入应用”的顺序说明首次部署。开始前，请准备与目标系统和 GPU 匹配的软件包，以及适用于该设备的有效 `license.key`。

2.1 运行与编译环境

下表列出当前 26.44 发行渠道。同一厂商的不同运行时不能互换，请根据包名、厂商工具包版本和 `glibc` 要求选择。

厂商	工具包／运行时	主机架构	说明
NVIDIA	CUDA 12.4、12.9、13.1	x86_64	固定入口根据 GPU 架构选择对应计算核心。
海光	DTK 24.04.3、25.04.4、26.04	x86_64	包内包含 gfx906 与 gfx936 目标。
AMD	ROCm 7.2.1	x86_64	HIP 运行时。
英特尔	oneAPI 2026.1	x86_64	SYCL 与 Level Zero 运行时。
摩尔线程	MUSA 5.1	x86_64	MUSA 运行时。
砺算	OpenCL 0.1.0	x86_64	OpenCL 运行时。
沐曦	HPCC 3.5.3.18	aarch64	HCR 运行时。

发行包 README 给出精确的 `glibc` 下限、CPU 架构、GPU 目标和依赖库。编译示例需要 C11 编译器和 GNU Make；使用 CMake 时要求 3.18 或更高版本。不要通过改写 ELF 版本标签或混用其他发行包的 `lib/` 来绕过版本检查。

2.2 解压下载文件

以下命令以 CUDA 12.4 包为例。其他平台只需替换压缩包名称。

```
mkdir -p "$HOME/xianzhikrylov-26.44"
tar -xzf xianzhikrylov-26.44-CUDA-CUDA-12.4-x86_64-glibc2.35.tar.gz \
  -C "$HOME/xianzhikrylov-26.44" --strip-components=1
export XK_ROOT="$HOME/xianzhikrylov-26.44"
```

压缩包根目录为 `XianzhiKrylov/`。解压后应保留 `include/`、`lib/` 和 `samples/` 的相对位置；NVIDIA 包中的 `lib/core*/` 是固定入口自动选择的架构核心，不能单独移动。

2.3 配置运行路径

设置包内库目录；NVIDIA 示例还需要 CUDA 运行库路径。其他厂商使用相应的运行时目录。

```
export LD_LIBRARY_PATH="$XK_ROOT/lib:/path/to/vendor/runtime${LD_LIBRARY_PATH:+:
    $LD_LIBRARY_PATH}"
```

应用与发行包必须使用同一套 GPU 运行库。只复制 `libXianzhiKrylov.so` 而遗漏同包的架构核心或依赖库，通常会导致启动或初始化失败。

2.4 配置 License

将下载的授权文件复制到解压后的包根目录：

```
cp /absolute/path/to/license.key "$XK_ROOT/license.key"
```

授权检查在本地完成，不需要网络连接。授权绑定产品、设备、允许的版本范围和有效期。设备不匹配、授权过期或构建完整性检查失败时，初始化或计算接口返回 XIANZHIKRYLOV_LICENSE_ERROR。

> **说明：授权与设备编号**
> xianzhikrylov_create() 的 device_id 是当前进程可见设备的编号，通常从 0 开始。固定节点或设备映射环境中的应用应先通过厂商工具确认实际可见设备，再分配与该设备匹配的授权和计算缓冲区。

2.5 编译与运行示例

进入 FGMRES + ILU(0) 样例目录，编译后运行：

```
cd "$XK_ROOT/samples/17_fgmres_ilu0"
make
make run
```

样例读取 matrices/transport_1m.csrpat，创建确定性矩阵和右端项，调用十函数 C 接口完成一次 PHASE_ALL 求解，并在 CPU 上重新计算原方程残差。正常输出的关键字段包括：

```
XianzhiKrylov status=1 converged=1 iterations=... method=... \
  residual=... rhs_norm=... relative_residual=...
Independent residual ... PASS
```

各样例都支持 make、makerun 和 makeclean。从 samples/ 根目录执行 make 可构建全部样例。

2.6 应用链接

应用至少包含 XianzhiKrylovAPI.h 和 XianzhiKrylovData.h。C11 程序的编译命令示例如下：

```
cc -std=c11 my_app.c \
  -I"$XK_ROOT/include" -L"$XK_ROOT/lib" \
  -Wl,-rpath,"$XK_ROOT/lib" \
  -lXianzhiKrylov -ldl -lm
```

若 GPU 运行库放在非系统目录，还需在运行环境中配置 LD_LIBRARY_PATH，或在链接阶段设置 -rpath-link。应用不需要直接链接厂商内部计算核心。

2.7 CMake 接入

源码包中的 examples/CMakeLists.txt 支持通过 XIANZHIKRYLOV_ROOT 指定软件包目录：

```
cmake -S examples -B build \
  -DXIANZHIKRYLOV_ROOT=/absolute/path/to/XianzhiKrylov
cmake --build build
```

CMake 使用同一目录中的 include/ 和 lib/，并把运行库搜索路径写入可执行程序。

3 通用说明

3.1 句柄与状态

xianzhikrylov_handle_t 保存设备、JSON 配置、准备结果、迭代工作区、最近一次求解统计和授权状态。矩阵和向量使用独立的不透明对象 xianzhikrylov_matrix_t 与 xianzhikrylov_vector_t。

推荐调用顺序如下：

1. xianzhikrylov_create()：创建绑定可见设备的计算句柄。
2. xianzhikrylov_config_set()：读取 JSON 配置并完成授权初始化。
3. xianzhikrylov_matrix_create()：创建矩阵描述对象。
4. xianzhikrylov_vector_create() 或多 RHS 版本：创建向量对象。
5. xianzhikrylov_execute()：执行分析、数值准备或求解阶段。
6. xianzhikrylov_print_info()：打印最近一次求解统计。
7. 销毁矩阵、向量和计算句柄。

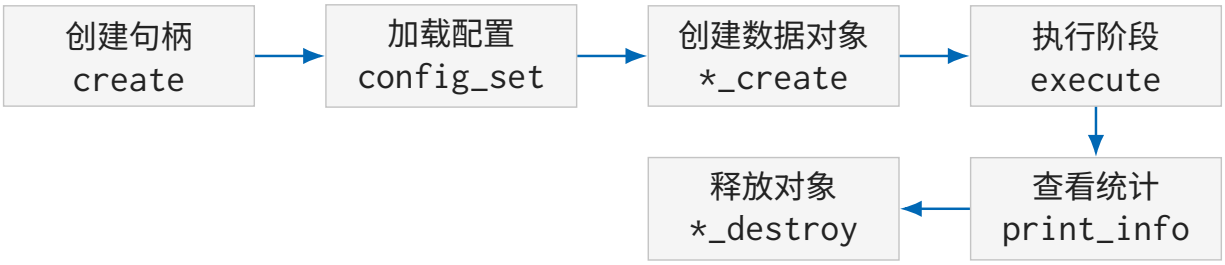


图 1: 推荐的调用顺序。普通用户可使用 ALL 阶段一次完成准备和求解。

3.2 执行阶段

xianzhikrylov_execute() 接收矩阵 A、解对象 x 和右端项对象 b，并按阶段编号执行：

阶段	行为与参数要求
XIANZHIKRYLOV_PHASE_ANALYSIS	使用 A 的结构信息建立规划；x、b 传 NULL。
XIANZHIKRYLOV_PHASE_NUMERIC_PREPARE	使用 A 的数值建立预条件器；x、b 传 NULL。
XIANZHIKRYLOV_PHASE_SOLVE	使用已准备的数据求解，A、x、b 均需提供。
XIANZHIKRYLOV_PHASE_ALL	自动完成分析、数值准备、求解并写入 x。

ANALYSIS 和 NUMERIC_PREPARE 完成时返回 XIANZHIKRYLOV_SUCCESS。SOLVE 和 ALL 只有在全部 RHS 均满足停止条件时返回 XIANZHIKRYLOV_CONVERGED。

3.3 内存所有权与生命周期

HOST 输入由库在 execute() 中准备到内部设备存储；DEVICE 输入借用调用方设备缓冲区。借用期间调用方必须保持地址和内容有效。

矩阵和向量对象不接管数组所有权。应用负责分配和释放主机或设备缓冲区，并在对象存活期间保持数组有效。销毁矩阵或向量对象只释放描述对象，不释放数组。

如果本次路线不能直接把结果发布到 DEVICE 解对象，则 execute() 返回 XIANZHIKRYLOV_UNSUPPORTED。HOST 解对象由库直接写入调用方缓冲区。

3.4 并发使用

同一计算句柄的调用必须串行。并行求解独立系统时，应为每个线程或任务使用独立计算句柄和独立数据对象。

3.5 配置与复用

求解方法、预条件器、容差和初值策略由 config_set() 加载的 JSON 确定。JSON 是算法配置的唯一来源，execute() 不接受运行时算法选项。

推荐复用规则：

- 矩阵和 RHS 不变、重复求新解：再次执行 SOLVE；
- 结构不变、矩阵数值改变：执行 NUMERIC_PREPARE，再执行 SOLVE；
- 结构改变：依次执行 ANALYSIS、NUMERIC_PREPARE、SOLVE；
- 不需要阶段拆分：执行 ALL。

外部 AMG 的 setup 在 NUMERIC_PREPARE 中完成，外层 Krylov 的每次预条件调用在 SOLVE 中执行一个外部 AMG cycle。

3.6 时间统计

xianzhikrylov_print_info() 打印最近一次求解的状态、各 RHS 的迭代次数、方法编号、残差、右端项范数和准备／求解时间。打印本身不重新求解。

数据读取、对象创建、结果复制、打印和销毁不计入准备／求解时间。阶段拆分时，ANALYSIS、NUMERIC_PREPARE 和 SOLVE 分别计量。

4 数据类型与存储格式

本章说明公共头文件中的对象、矩阵描述和向量布局。所有索引从 0 开始，数组按主机本机字节序存储。接口不执行隐式格式转换、排序、去重、索引宽度转换或精度转换。

4.1 不透明对象

```
typedef struct xianzhikrylov_handle_opaque *xianzhikrylov_handle_t;
typedef struct xianzhikrylov_matrix_opaque *xianzhikrylov_matrix_t;
typedef struct xianzhikrylov_vector_opaque *xianzhikrylov_vector_t;
```

应用只保存这些指针，不读取或修改内部字段。矩阵和向量对象是数据描述对象，不是额外的计算句柄。

4.2 数值类型

常量	C 存储类型	说明
XIANZHIKRYLOV_FP64	double	标准实数方法。
XIANZHIKRYLOV_FP32	float	原生单精度方法。
XIANZHIKRYLOV_COMPLEX_FP64	xianzhikrylov_complex_t	实部、虚部各为 double。
XIANZHIKRYLOV_COMPLEX_FP32	xianzhikrylov_complex_fp32_t	实部、虚部各为 float。
XIANZHIKRYLOV_DD	两个 double 平面	Double-Double。

是否支持某一类型由发行包、矩阵格式、方法和预条件器共同决定。不支持时返回 XIANZHIKRYLOV_UNSUPPORTED，库不会静默改用其他精度。

4.3 矩阵描述

xianzhikrylov_matrix_data_t 的第一个字段为 ABI 版本和结构大小：

```
memset(&data, 0, sizeof(data));
data.abi_version = XIANZHIKRYLOV_ABI_VERSION;
data.struct_size = sizeof(data);
```

当前 XIANZHIKRYLOV_ABI_VERSION 为 3。

公共字段包括：

- 矩阵格式、数值类型、索引类型和存储空间；
- 行数、列数和存储项数；
- BSR 块大小和块内布局；
- 行指针、列索引、对角位置和数值缓冲区；
- MSR 非对角数值；
- LDU 上下三角地址和面系数；
- Double-Double 对应的低位平面。

未使用的格式缓冲区必须同时满足 data=NULL 和 bytes=0。应用必须保持所有引用的数组在相关 execute() 调用期间有效。

4.4 稀疏存储格式

4.4.1 CSR

CSR 使用 row_offsets、column_indices 和 values。rows 为标量行数，columns 为标量列数，entries 为非零元数。每行列号必须严格递增、无重复并位于 $[0, n)$。CSR 要求每行包含对角项。

4.4.2 BSR

BSR 的 rows、columns 和 entries 都按块计数。块值按 XIANZHIKRYLOV_BLOCK_ROW_MAJOR 或 XIANZHIKRYLOV_BLOCK_COLUMN_MAJOR 存放。索引对应块行列；常用块大小为 2、4 或 5。

4.4.3 MSR

MSR 使用非对角行指针和列索引，values 保存每个标量行的对角值，offdiagonal_values 保存非对角值。entries 表示非对角块数量。

4.4.4 LDU

LDU 使用 values 保存对角值，lower_address 和 upper_address 保存每个内部面的两端行号，lower_values 和 upper_values 保存面系数。地址使用 int32_t。

4.5 向量与多右端项

单 RHS 接口创建连续、RHS-major、步长等于长度的向量。多 RHS 接口支持：

- length：标量行数；
- rhs_count：列数；
- leading_dimension：步长；
- layout：RHS-major 或 row-major；
- data：主数据平面；
- low：Double-Double 低位平面，普通类型传 NULL。

布局	元素 (i, j) 的地址
XIANZHIKRYLOV_RHS_MAJOR	data[j*leading_dimension+i]；步长不小于 length。
XIANZHIKRYLOV_ROW_MAJOR	data[i*leading_dimension+j]；步长不小于 rhs_count。

4.6 Double-Double

DD 数学值等于高位与低位之和。矩阵低位存放在 values_low、offdiagonal_values_low 或 LDU 对应的 *_values_low 字段；RHS、初值和解同时使用 data 与 low。接口不做隐式 DD 到 FP64 舍入。

4.7 检查清单

建立对象前检查：

1. `abi_version` 和 `struct_size` 正确；
2. 类型、格式、块大小、索引宽度和存储空间匹配；
3. 每行列号合法、严格递增、无重复；
4. 所有缓冲区字节数足够；
5. 设备对象属于计算句柄绑定的设备；
6. 输入中不存在意外的 NaN、Inf 或不可用对角项。

5 函数参考

公开接口包含十个函数，所有函数返回 xianzhikrylov_status_t。XIANZHIKRYLOV_SUCCESS 表示动作完成；XIANZHIKRYLOV_CONVERGED 表示求解阶段满足停止条件。

函数	用途
xianzhikrylov_create()	创建绑定设备的计算句柄。
xianzhikrylov_config_set()	加载求解 JSON 和授权配置。
xianzhikrylov_matrix_create()	创建矩阵描述对象。
xianzhikrylov_vector_create()	创建连续单 RHS 向量。
xianzhikrylov_multiple_vector_create()	创建多 RHS、显式布局或 DD 向量。
xianzhikrylov_execute()	执行分析、数值准备或求解阶段。
xianzhikrylov_print_info()	打印最近一次求解统计。
xianzhikrylov_matrix_destroy()	销毁矩阵对象。
xianzhikrylov_vector_destroy()	销毁向量对象。
xianzhikrylov_destroy()	销毁计算句柄。

5.1 xianzhikrylov_create()

```
xianzhikrylov_status_t xianzhikrylov_create(
    int32_t device_id,
    xianzhikrylov_handle_t *handle);
```

参数	方向	说明
device_id	输入	当前进程可见设备编号，通常为 0。
handle	输出	成功时接收计算句柄；失败时置为 NULL。

5.2 xianzhikrylov_config_set()

```
xianzhikrylov_status_t xianzhikrylov_config_set(
    xianzhikrylov_handle_t handle,
    xianzhikrylov_config_option_t option,
    const char *value);
```

参数	方向	说明
handle	输入	已成功创建的计算句柄。
option	输入	ABI 3 当前为 XIANZHIKRYLOV_CONFIG_FILE。
value	输入	以空字符结尾的 JSON 文件路径。

函数读取 JSON、选择方法／预条件器并完成配置阶段授权检查。必须在首次执行依赖配置的 execute() 之前调用。库复制路径字符串，不保留调用方指针。

5.3 xianzhikrylov_matrix_create()

```
xianzhikrylov_status_t xianzhikrylov_matrix_create(
    xianzhikrylov_matrix_t *matrix,
    const xianzhikrylov_matrix_data_t *data);
```

参数	方向	说明
matrix	输出	成功时接收矩阵对象。
data	输入	ABI 版本、格式、类型、尺寸和数组描述。

该函数只创建描述对象，不执行完整设备分配、结构分析或预条件器构建。矩阵数组由调用方管理。

5.4 xianzhikrylov_vector_create()

```
xianzhikrylov_status_t xianzhikrylov_vector_create(
    xianzhikrylov_vector_t *vector,
    int64_t size,
    void *data,
    xianzhikrylov_value_type_t value_type,
    xianzhikrylov_memory_space_t memory_space);
```

参数	方向	说明
vector	输出	成功时接收向量对象。
size	输入	元素个数，应等于矩阵阶数。
data	输入/输出	连续数据缓冲区；用于解对象时由库写入结果。
value_type	输入	向量的数值类型。
memory_space	输入	HOST 或 DEVICE。

单向量采用紧凑 RHS-major 布局。Double-Double 向量必须使用多向量接口以提供低位平面。

5.5 xianzhikrylov_multiple_vector_create()

```
xianzhikrylov_status_t xianzhikrylov_multiple_vector_create(
    xianzhikrylov_vector_t *vector,
    int64_t length,
    int64_t rhs_count,
    int64_t leading_dimension,
    xianzhikrylov_layout_t layout,
    void *data,
    void *low,
    xianzhikrylov_value_type_t value_type,
    xianzhikrylov_memory_space_t memory_space);
```

参数	方向	说明
vector	输出	成功时接收向量对象。
length	输入	每个 RHS 的元素数。
rhs_count	输入	右端项列数，至少为 1。
leading_dimension	输入	列间或行间跨度，按完整数值元素计数。
layout	输入	XIANZHIKRYLOV_RHS_MAJOR 或 XIANZHIKRYLOV_ROW_MAJOR。
data	输入/输出	主数据平面。
low	输入/输出	DD 低位平面；普通类型传 NULL。
value_type	输入	数值类型。
memory_space	输入	HOST 或 DEVICE。

5.6 xianzhikrylov_execute()

```
xianzhikrylov_status_t xianzhikrylov_execute(
    xianzhikrylov_handle_t handle,
    xianzhikrylov_phase_t phase,
    xianzhikrylov_matrix_t matrix,
    xianzhikrylov_vector_t solution,
    xianzhikrylov_vector_t rhs);
```

参数	方向	说明
handle	输入	已成功配置的计算句柄。
phase	输入	分析、数值准备、求解或完整执行阶段。
matrix	输入	系数矩阵对象。
solution	输出数据	分析／数值准备阶段为 NULL；求解阶段为解对象。
rhs	输入	分析／数值准备阶段为 NULL；求解阶段为右端项对象。

阶段	参数与结果
PHASE_ANALYSIS	只要求 matrix；返回 SUCCESS。
PHASE_NUMERIC_PREPARE	只要求 matrix；返回 SUCCESS。
PHASE_SOLVE	要求 A、x、b；全部 RHS 收敛时返回 CONVERGED。
PHASE_ALL	自动完成准备和求解；全部 RHS 收敛时返回 CONVERGED。

5.7 xianzhikrylov_print_info()

```
xianzhikrylov_status_t xianzhikrylov_print_info(
    xianzhikrylov_handle_t handle);
```

打印最近一次求解的状态、每 RHS 迭代次数、方法编号、残差、右端项范数和准备／求解时间。该函数不重新求解。

5.8 xianzhikrylov_matrix_destroy()

```
xianzhikrylov_status_t xianzhikrylov_matrix_destroy(
    xianzhikrylov_matrix_t matrix);
```

销毁矩阵描述对象，不释放应用提供的数组。成功后应用应将保存对象的变量置为 NULL。

5.9 xianzhikrylov_vector_destroy()

```
xianzhikrylov_status_t xianzhikrylov_vector_destroy(
    xianzhikrylov_vector_t vector);
```

同时适用于单 RHS 和多 RHS 向量，不释放数据缓冲区。

5.10 xianzhikrylov_destroy()

```
xianzhikrylov_status_t xianzhikrylov_destroy(
    xianzhikrylov_handle_t handle);
```

释放句柄拥有的配置、工作区、准备结果和外部 provider 状态。销毁不要求当前授权仍然有效。

6 使用示例

本章以发行包中的 samples/17_fgmres_ilu0 为上手样例，再说明 JSON 配置、阶段拆分、相同结构更新和多右端项。该样例使用主机端 CSR 矩阵、RHS 和解缓冲区，完整源码、配置、矩阵生成规则和构建文件均随发行包提供。

6.1 快速上手

样例求解 transport_1m.csrpat 描述的百万阶 CSR 稀疏模式。程序从固定种子生成矩阵数值和右端项，调用十函数接口完成一次 FGMRES + ILU(0) XIANZHIKRYLOV_PHASE_ALL 求解，然后在主机端独立计算原系统残差。接口调用顺序与 samples/17_fgmres_ilu0/sample.c 完全一致：

```
status = xianzhikrylov_create(0, &handle);

status = xianzhikrylov_config_set(
    handle, XIANZHIKRYLOV_CONFIG_FILE, "config.json");

status = xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_ALL,
    matrix, solution, rhs);

status = xianzhikrylov_print_info(handle);
status = xianzhikrylov_destroy(handle);
```

其中矩阵对象与两个向量对象已由样例公共辅助代码创建；完整调用还包括 xianzhikrylov_matrix_create() 和 xianzhikrylov_multiple_vector_create()。编译和运行：

```
XK_ROOT=/absolute/path/to/XianzhiKrylov
cd "$XK_ROOT/samples/17_fgmres_ilu0"
make
make run
```

包根目录必须包含 license.key。正常结束后，程序打印残差、迭代次数和计时，并输出 PASS；详细说明见该目录的 README.md。

6.2 JSON 配置

```
{
  "schema_version": 1,
  "matrix_format": "csr",
  "block_size": 1,
  "solver": {
    "method": "fgmres",
    "preconditioner": "ilu0",
    "max_iterations": 2000,
    "restart": 40,
    "relative_tolerance": 1e-9,
    "absolute_tolerance": 0,
    "initial_guess": "zero"
  }
}
```

字段	说明
schema_version	普通配置必填，当前为 1。
matrix_format	auto/csr/bsr/msr/ldu。
block_size	标量 CSR 使用 1；块矩阵使用对应块大小。
solver.method	cg/pcg/fcg/bicg/bicgstab/gmres/fgmres/gcr。
solver.preconditioner	none/identity/jacobi/ilu0/ic0 或发行包开放的其他预条件器。
solver.max_iterations	正整数，总迭代上限。
solver.restart	GMRES/FGMRES/GCR 的重启长度。
solver.relative_tolerance	非负相对容差。
solver.absolute_tolerance	非负绝对容差。
solver.initial_guess	zero/provided/reuse。
residual_norm	l1/l2/linf（可选），停止和报告使用同一范数。

6.3 常用配置：FGMRES + ILU(0)

```
{
  "schema_version": 1,
  "matrix_format": "csr",
  "block_size": 1,
  "solver": {
    "method": "fgmres",
    "preconditioner": "ilu0",
    "max_iterations": 2000,
    "restart": 40,
    "relative_tolerance": 1e-9,
    "absolute_tolerance": 0,
    "initial_guess": "zero",
    "residual_norm": "l2"
  }
}
```

方法、预条件器、容差和初值策略只来自 JSON。需要更换求解方法时，应销毁句柄并用新配置重新创建，不能在 execute() 中临时切换。

6.4 分阶段执行

如果应用需要分别统计分析和数值准备时间：

```
xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_ANALYSIS,
    matrix, NULL, NULL);

xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_NUMERIC_PREPARE,
    matrix, NULL, NULL);

xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_SOLVE,
```

```
    matrix, solution, rhs);
```

ANALYSIS 和 NUMERIC_PREPARE 成功时返回 SUCCESS；SOLVE 只有在全部 RHS 收敛时才返回 CONVERGED。

6.5 相同稀疏结构，矩阵数值改变

保持矩阵对象、格式和数组地址不变，更新数值缓冲区，然后执行：

```
xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_NUMERIC_PREPARE,
    matrix, NULL, NULL);

xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_SOLVE,
    matrix, solution, rhs);
```

如果稀疏结构也改变，应先重新执行 ANALYSIS。

6.6 同一矩阵，连续更新右端项

矩阵、解对象地址和布局保持不变，更新 RHS 数据后再次执行 SOLVE：

```
xianzhikrylov_execute(
    handle, XIANZHIKRYLOV_PHASE_SOLVE,
    matrix, solution, rhs);
```

已建立的结构和数值准备可复用时，无需再次执行准备阶段。

6.7 提供非零初值

JSON 中设置 "initial_guess":"provided"，并在求解前把初值写入 solution 对应的数据缓冲区。求解完成后，库用最终解覆盖该缓冲区。

```
{
  "solver": {
    "initial_guess": "provided"
  }
}
```

"zero" 表示忽略解缓冲区原内容并从零初值开始；"reuse" 使用方法支持的上一次内部解。

6.8 多右端项

使用 xianzhikrylov_multiple_vector_create()：

```
xianzhikrylov_multiple_vector_create(
    &rhs, n, nrhs, ld,
    XIANZHIKRYLOV_RHS_MAJOR,
    b, NULL, XIANZHIKRYLOV_FP64,
    XIANZHIKRYLOV_HOST);

xianzhikrylov_multiple_vector_create(
    &solution, n, nrhs, ld,
    XIANZHIKRYLOV_RHS_MAJOR,
```

```
    x, NULL, XIANZHIKRYLOV_FP64,
    XIANZHIKRYLOV_HOST);
```

每个 RHS 独立检查收敛；只有全部列满足条件时，`execute()` 才返回 `CONVERGED`。

6.9 外部 AMG 预条件器

发行包支持时，可在 JSON 中选择 LinglongAMG：

```
{
  "preconditioner": {
    "type": "amg",
    "provider": "linglongamg",
    "library_path": "/path/to/liblinglongamg.so",
    "config_path": "/path/to/amg_config.json"
  }
}
```

外部 provider 的 setup 在 `NUMERIC_PREPARE` 中完成；外层 Krylov 的每次预条件调用在 `SOLVE` 中执行一个 AMG cycle。外部库和 XianzhiKrylov 分别进行授权检查。

7 错误处理与结果验证

API 状态只说明调用是否完成，数值结果质量需要另行验证。应用应记录第一次失败接口的状态、输入类型、设备编号和原始诊断，再使用原始 A、b 检查返回解。

7.1 错误状态

值	常量	含义与处理
0	XIANZHIKRYLOV_SUCCESS	API 操作成功完成。
1	XIANZHIKRYLOV_CONVERGED	所有右端项满足求解停止条件。
2	XIANZHIKRYLOV_MAX_ITERATIONS	达到迭代上限；检查上限、预条件器和方法适用性。
3	XIANZHIKRYLOV_BREAKDOWN	迭代过程中出现数学崩溃。
4	XIANZHIKRYLOV_INVALID_ARGUMENT	参数、尺寸、布局、容量或描述符 ABI 无效。
5	XIANZHIKRYLOV_UNSUPPORTED	方法、格式、精度或内存空间组合未开放。
6	XIANZHIKRYLOV_STALE_SYSTEM	系统状态已失效，需要重新导入或准备。
7	XIANZHIKRYLOV_DEVICE_ERROR	GPU 运行时或设备执行失败。
8	XIANZHIKRYLOV_STATE_ERROR	句柄状态不允许当前调用。
9	XIANZHIKRYLOV_RESIDUAL_MISMATCH	结果残差与期望停止条件不一致。
10	XIANZHIKRYLOV_STAGNATED	迭代停滞，结果不能按收敛处理。
11	XIANZHIKRYLOV_LICENSE_ERROR	授权、设备、期限或构建完整性检查失败。

不要把 2、3、9、10 或 11 当作收敛。若接口返回失败，后续依赖该状态的计算应停止。

7.2 常见集成问题

现象	检查方向
找不到头文件	使用与发行包一致的 include/；不要混用其他版本头文件。
配置或求解返回 11	检查 license.key 是否存在、设备是否匹配、授权是否过期。
启动时缺共享库	检查 LD_LIBRARY_PATH、包内架构核心和厂商运行库。
config_set 返回 4	检查 JSON 路径、文件内容和配置字段。
matrix_create 返回 4	检查 ABI、格式、类型、索引宽度、尺寸和缓冲区字节数。
vector_create 返回 4	检查长度、RHS 数、步长、布局和类型。
execute 返回 5	检查当前发行包是否支持该格式、精度、方法和阶段组合。
execute 未收敛	检查右端项、初值、容差、最大迭代和预条件器适用性。

7.3 数值精度与残差

设 $r = b - Ax$。当前支持的残差范数为：

- l1：$\|r\|_1 = \sum_i |r_i|$；

- `l2`：$\|r\|_2 = \sqrt{\sum_i |r_i|^2}$；
- `linf`：$\|r\|_\infty = \max_i |r_i|$。

停止条件为：

$$\|r\|_p \le \max\left(atol,\ rtol\|b\|_p\right).$$

因此，绝对容差和相对容差不需要同时分别成立，而是取两个阈值中较大的一个作为本次停止阈值。返回的残差和归一化分母必须使用同一范数。

注意：范数只改变停止判据

选择 L1 或 Linf 不会把所有内部数学运算替换成相应范数。例如，GMRES/FGMRES 的 Arnoldi 正交化仍使用算法需要的 L2 内积和范数；配置的 `residual_norm` 只影响外部收敛判断和报告。

7.3.1 独立验证

应用应根据原始系统重新计算 $r = b - Ax$，不要仅依赖库内统计。推荐记录：

- 矩阵阶数、非零元数、格式和精度；
- 右端项数量、布局、步长和范数；
- 方法、预条件器、容差、重启长度、实际迭代次数；
- 最大绝对残差、所选范数残差和相对残差；
- 是否所有 RHS 均收敛，以及失败 RHS 的编号。

高精度或病态矩阵建议使用更高精度的独立实现计算残差。Double-Double 解应使用高、低位组合后验证，不应只检查高位平面。

7.4 问题报告

检查输入和结果中是否存在 NaN、Inf、极端缩放、零对角或明显错误的向量长度。应用可以设置自己的质量门槛，将“API 已收敛”和“业务验收通过”分开记录。不得为了让失败结果通过检查而修改右端项或放宽容差。

附录 A　示例输入文件与随包样例

发行包的 `samples/` 目录提供可独立编译的 C 样例。每个样例包含 `sample.c`、`config.json`、`Makefile` 和 `README.md`；公共矩阵、辅助代码和构建规则位于上层目录。

A.1　随包样例

方法类别	主要样例	说明
无预条件	CG、BiCG、BiCGStab、GMRES、FGMRES、GCR	展示方法与迭代控制。
Jacobi	PCG、FCG、BiCG、BiCGStab、GMRES、FGMRES、GCR	CSR 实数方法。
ILU(0)	BiCG、BiCGStab、GMRES、FGMRES、GCR	非对称一般矩阵。
IC(0)	PCG、FCG、GMRES、FGMRES、BiCGStab、BiCG、GCR	对称正定或适用矩阵。
BSR	FGMRES + ILU(0)、块 Jacobi、多色块 DILU	块结构矩阵。
MSR	BiCGStab + Jacobi	对角分离存储。
LDU	PCG + DIC、BiCGStab + DILU	面结构存储。
扩展精度	complex FP64 批量、原生 FP32、complex FP32、DD	仅在发行包包含对应样例时使用。

A.1.1　构建规则

每个样例目录均支持：

```
make
make run
make clean
```

根目录执行 `make` 会构建当前目录中发现的全部样例。包内样例通过相对链接找到包根目录的 `include/` 和 `lib/`，因此不要单独移动样例目录。

A.2　矩阵：XKPAT1 模式格式

样例矩阵使用 XKPAT1 格式。文件只保存稀疏模式和固定生成参数，不保存数值或右端项。文件头为 64 字节，整数采用小端序。

偏移	类型	含义

0	8 字节	魔数 XKPAT1；不足部分补零。
8	uint32_t	格式版本，当前为 1。
12	uint32_t	1 表示实数 FP64，2 表示复数 FP64。
16	uint64_t	标量矩阵阶数 n。
24	uint64_t	存储项数 nnz。
32	uint32_t	右端项列数。
36	uint32_t	建议块大小。
40	uint32_t	模型编号。
44	uint32_t	保留字段，必须为 0。
48	uint64_t	固定值生成种子。
56	uint64_t	保留字段，必须为 0。

文件头之后依次存放 int32_trow_offsets[n+1] 和 int32_tcolumn_indices[nnz]。行指针首项为 0、末项为 nnz，每行列号严格递增且无重复。

样例根据结构、固定种子和模型编号确定性生成矩阵值。实数系统的已知解为 $x = (1, \ldots, 1)^{\mathsf{T}}$，右端项由 $b = A_{\text{generated}} x$ 生成。求解仍从 JSON 指定初值开始，已知解只用于构造 b 和独立残差验证。

A.2.1 矩阵模型

编号	模型	结构特征
1	diffusion_1m	对称正定、严格对角占优。
2	transport_1m	一般非对称、正行和。
3	coupled4_500k	对称块型、严格对角占优。
4	complex_262k_3rhs	严格对角占优复数矩阵，三 RHS。

模型文件不包含用户矩阵的数值。应用使用自有矩阵时，应直接构造 xianzhikrylov_matrix_data_t，并通过矩阵和向量创建函数建立对象，不需要转换为 XKPAT1。

附录 B　文档与支持

B.1　交付文件

文件或目录	用途
`README.md`	发行包说明、运行环境、安装、样例和接口使用指南。
`include/XianzhiKrylovAPI.h`	七个公开 C 函数和状态常量。
`include/XianzhiKrylovData.h`	矩阵、向量、解和系统描述结构。
`lib/`	公共入口、平台核心和依赖关系检查所需的运行文件。
`samples/`	可编译样例、JSON 配置、矩阵模式和验证程序。
`THIRD_PARTY_NOTICES.txt`	第三方组件许可与归属说明。

B.2　技术支持

软件包与 License 下载地址：

https://store.solved-tech.com/

请求技术支持时，请提供：

- 产品版本、库版本、平台包文件名和 SHA-256；
- CPU、glibc、GPU 型号和厂商工具包版本；
- 初始化、系统接入、准备和求解各步骤的返回状态；
- JSON 配置、矩阵规模、格式、精度和 RHS 数量；
- 独立残差和最简复现程序。

B.2.1　企业微信

可用企业微信扫描末页二维码联系技术支持。请勿通过公开渠道发送真实密钥、生产设备序列号或受限制矩阵。

附录 C　版本日志

本章按从新到旧列出产品的主要接口变化。

XianzhiKrylov 26.44

- 公共接口更新为 ABI 3 十函数 C API。
- `create` 只创建计算句柄；JSON 由 `config_set` 加载。
- 增加矩阵、单 RHS 向量和多 RHS 向量对象创建／销毁函数。
- 增加 `xianzhikrylov_execute()`，统一执行分析、数值准备、求解和完整流程。
- 求解结果直接写入调用方解对象，不再提供下载或结果复制接口。
- 矩阵和向量对象使用不透明类型，数据数组所有权仍归调用方。
- Samples 与产品说明文档统一使用 ABI 3 调用方式。

XianzhiKrylov 26.43

- 统一七函数 C API，厂商由已安装发行包决定。
- JSON 选择方法、预条件器、容差和初值策略。
- 增加边界授权和统计信息接口。
- Samples 覆盖主要 Krylov 方法、预条件器、矩阵格式和数值类型。

XianzhiKrylov 26.42

- 统一开发者预览接口。
- 提供 CSR、BSR、MSR、LDU 和多种 Krylov/预条件器组合。